

Can the SPO tree find planted regimes?

A controlled test with a known answer

18 September 2026

In short. We build an artificial market in which one feature x decides the regime of an asset: its price drifts up while x is above a threshold d and down while it is below. We hide x among three useless features and ask the tree to trade the asset against cash. Because we planted the regimes ourselves, we know what a perfect learner should find, and we can measure how close the tree comes.

Results.

- With one decision per step, where the right answer is d , the tree finds x in 100 datasets of 100 and puts its threshold within 0.1 of d in 95 of them. It never uses a decoy.
- With one decision every 10 steps the right target is no longer d : the position is held while the regime keeps changing, so the threshold that matters lies below d (Section 3). The tree then splits first on x in 86 datasets of 100, never first on a decoy, and keeps no split in 14. Its thresholds lie between about -1.5 and 0 , around the zone where the decision changes.
- On test data the tree earns almost what the ideal rule earns (Sharpe ratio per decision 0.72 against 0.74, and 0.63 for always holding the asset).
- The threshold is found as long as a regime lasts about one holding period or more. When the feature is much faster than the holding period there is nothing left to gain, and the tree rightly keeps no split.
- *Caution:* the planted drift is very large (3% per step for a volatility of 3.2%). This test shows that the mechanism works; it does not show that weak, market-sized signals can be found.

1 The method, in the vocabulary of the articles

- **Predict, then optimize.** A predictor gives a return vector $\hat{c}$; a portfolio optimizer turns it into weights. Here the optimizer is the mean–variance problem with proportional transaction costs of Wang and Hasuiké (their “DIR” article, equation 8): maximise $\hat{c}^\top w - f^\top |w - u| - \lambda w^\top \Sigma w$ over long-only, fully invested weights w , where u are the current holdings. There are two assets: the risky asset and cash (constant price, no fee). The fee is 3 basis points and $\lambda = 1$.
- **SPO loss.** The predictor is not trained to forecast returns well. It is trained on the *decision regret*: the utility lost by trading on $\hat{c}$ instead of on the returns that really happened (the SPO loss of Elmachetoub and Grigas, with costs and risk inside, as in DIR equation 10).
- **SPO tree.** The predictor is a decision tree, as in “SPOT” (Elmachetoub, Liang and McNellis): each leaf holds one vector $\hat{c}$, and splits are chosen to reduce the SPO loss. Two things are specific to this implementation. The tree is trained on the path of its own past holdings, because costs depend on them. And, as SPOT does, it is pruned on validation data: the tree is grown on the first three quarters of the training decisions, and a split stays only if it lowers the regret *and* raises the Sharpe ratio on the last quarter.

- **Two clocks.** Data arrive at every step; the portfolio is decided every 10 steps and held in between, the way one would trade weekly on daily data. A training row is one decision: the features at that step, and the return of the asset over the next 10 steps.

2 The planted world

The data come from `synthetic_data.py`. The feature is a mean-reverting (Ornstein–Uhlenbeck) process, and the asset is a geometric random walk whose drift depends on it:

$$x_{s+1} = m + e^{-k}(x_s - m) + \text{noise}, \quad \log \frac{S_{s+1}}{S_s} = \begin{cases} +\mu - \frac{1}{2}\sigma^2 + \sigma\varepsilon_s & \text{if } x_s \geq d, \\ -\mu - \frac{1}{2}\sigma^2 + \sigma\varepsilon_s & \text{if } x_s < d. \end{cases}$$

Parameter	Base value	Meaning
μ	3% per step	drift of the asset: $+\mu$ above the threshold, $-\mu$ below
σ	3.2% per step	volatility of the asset (variance 0.001)
d	0	the planted threshold on x
m , variance of x	1, 4	long-run mean and variance of x : the threshold sits half a standard deviation below the mean, so x is above d in 69% of the steps
k	0.2 per step	reversion speed: a regime lasts about 5 steps on average
decision interval	10 steps	a position is held for 10 steps

Table 1: Base parameters (the defaults of `Case` in `recovery_test.py`).

The tree sees four features at each decision: x ; a 20-step moving average of x (informative, but a blurred and late copy); an unrelated mean-reverting process with the same law as x ; and white noise. A dataset has 443 training decisions (332 to grow the tree, 111 to prune it) followed by test decisions that are never used in training.

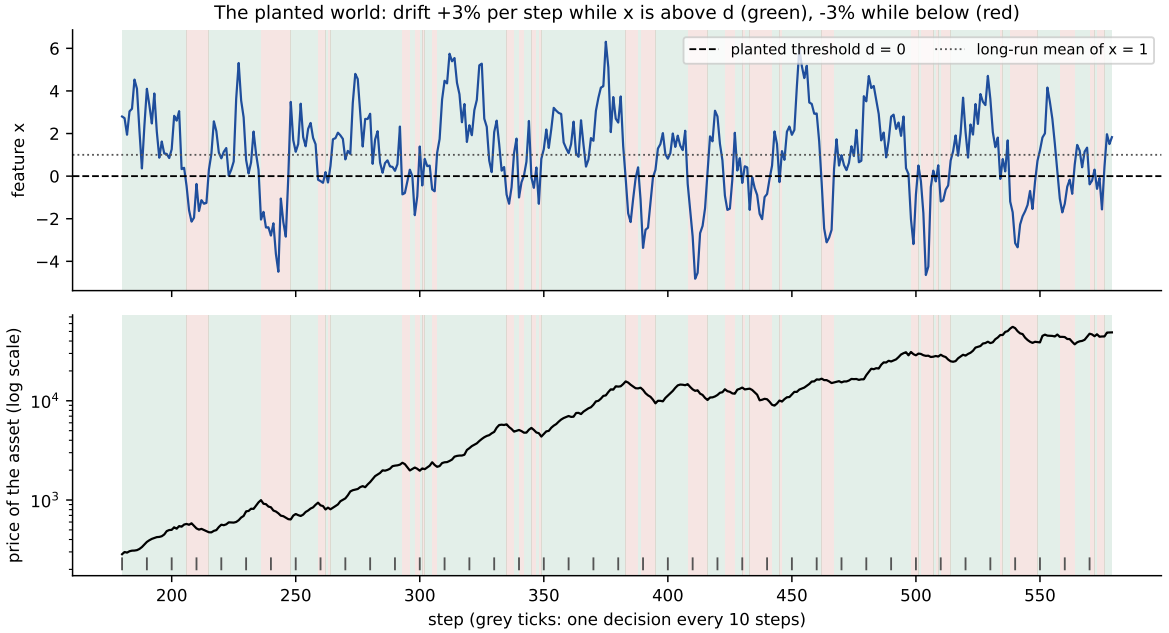


Figure 1: A stretch of one dataset. The background colour is the true regime, which the tree never sees. A decision is taken at each grey tick and held until the next one, while the regime keeps changing.

3 What should a correct tree find? Not d

This is the point that needs care. The tree decides once and holds for 10 steps, and during these 10 steps x keeps moving back towards its mean $m = 1$, which lies above d . So what matters at a decision is not the regime *now* but the expected return over the *next 10 steps*, given x now. With B the number of those steps spent above d minus the number spent below, this expected return is exactly

$$g(x) = \mathbb{E} \left(e^{\mu B} \mid x \right) - 1,$$

(the price noise averages out) and we compute it by simulation: the blue curve of Figure 2.

- g changes sign near $x \approx -0.9$, well *below* $d = 0$: just under d the first step is bad, but x soon climbs back above d and the next steps are good.
- The ideal position (lower panel) is zero below that point and rises to a full position where g reaches $2\lambda\sigma_{10}^2 \approx 3.6\%$ ($\sigma_{10} \approx 13\%$ is the volatility of the asset over 10 steps), near $x \approx -0.3$. Above that, the portfolio is fully invested.
- At d itself g jumps from about 5% to about 12%, but the decision is the same on both sides: fully invested. **A tree trained on decisions has no reason to split at d .** This is the SPO idea at work: the loss rewards finding where the *decision* changes, not where the conditional mean changes.

So the right question is not “does the tree find d ?” but “does the tree put its threshold where the decision changes, between about -0.9 and -0.3 ?”. The tree finds d itself in two cases: when a decision is taken at every step, or when d equals the mean of x (by symmetry). We test the first case below.

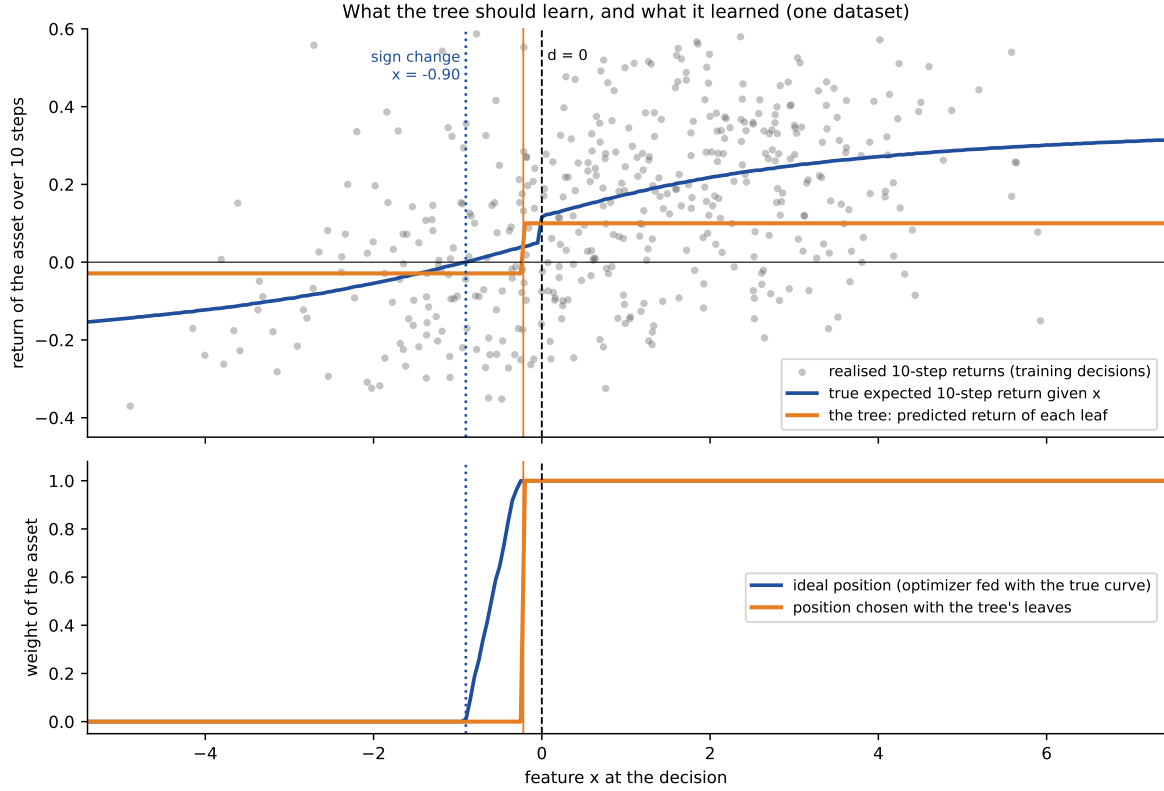


Figure 2: One dataset. *Top:* every dot is one training decision; the blue curve is the true expected 10-step return given x ; the orange steps are the two leaves of the fitted tree. *Bottom:* the position that the optimizer takes with the true curve (blue) and with the tree’s leaves (orange). Vertical lines: planted d (dashed), sign change of the true curve (dotted), the tree’s threshold (orange). The upper leaf sits at +10%, the largest value the search allows: leaf values are inputs to the optimizer, not forecasts, and any value above 3.6% gives the same fully invested portfolio.

4 Results

4.1 One dataset

For the dataset of Figures 1 and 2, growth made a single split, on x at -0.22 . Pruning kept it: on the 111 checking decisions it lowers the regret from 0.040 to 0.027 and raises the Sharpe ratio per decision from 0.62 to 0.74. No decoy is used. The lower leaf says “cash” (predicted return -2.9%), the upper leaf “fully invested”. The threshold sits at the top of the zone where the ideal position rises from 0 to 1.

1,000 test decisions	mean return	Sharpe ratio	turnover
Tree	13.9%	0.73	0.76
Ideal rule (optimizer fed with the true g)	14.0%	0.72	0.66
Rule at d (the asset while $x \geq d$, else cash)	13.6%	0.72	0.82
Always the asset (also the tree without splits)	12.6%	0.57	0.00

Table 2: One dataset; all figures are per decision (10 steps). The returns are huge because the planted drift is huge; only the comparison between the lines matters.

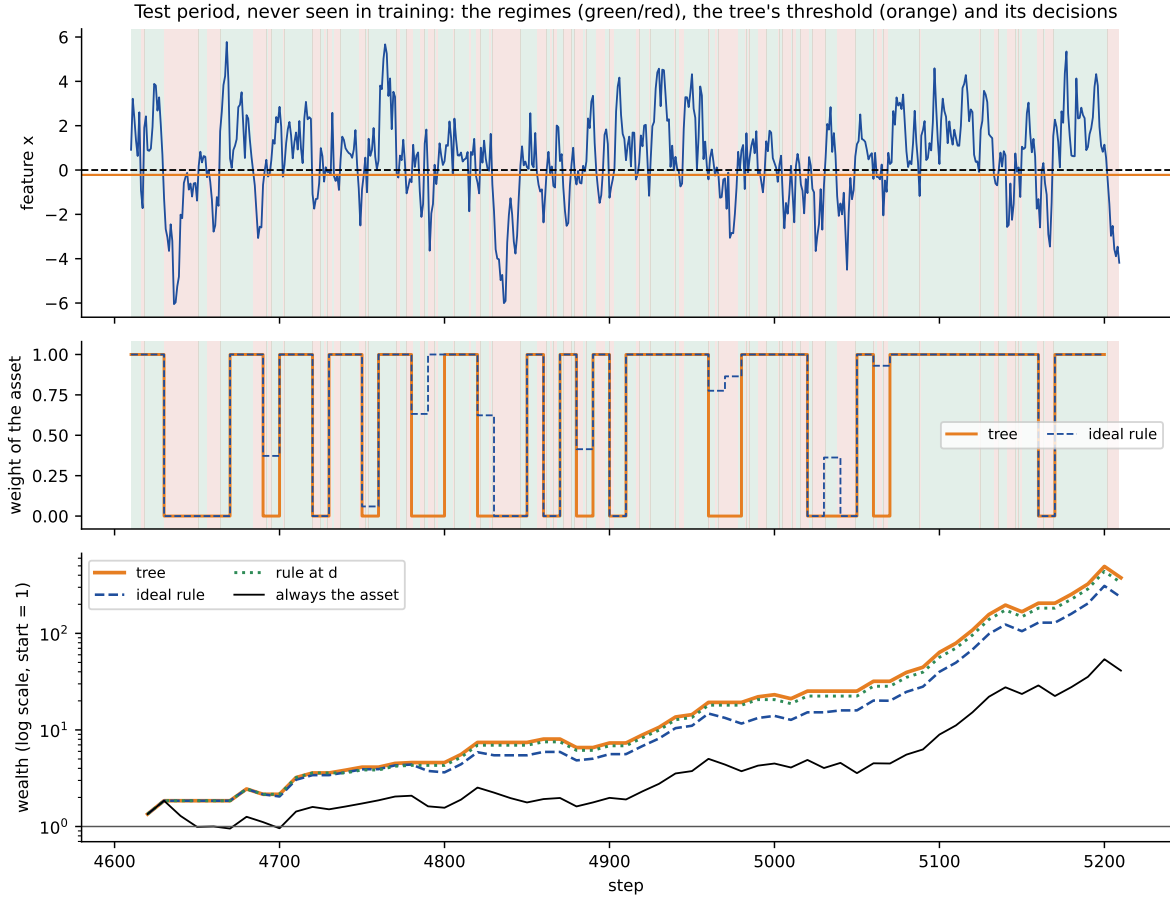


Figure 3: The first 60 test decisions of the same dataset. The tree (orange) goes to cash when x is below its threshold and is fully invested otherwise; the ideal rule (dashed blue) differs only by taking partial positions in the narrow band where the expected return is small.

4.2 Where the threshold lands, over 100 datasets

One decision per step (left panel of Figure 4). Here theory says the right threshold is exactly d . The first split is on x in 100 datasets of 100, every tree has exactly one split, and no decoy appears. The median threshold is 0.000; half of the thresholds lie within $[-0.015, +0.010]$ and 95 of 100 within 0.1 of d (the standard deviation of x is 2). On test data the tree has a Sharpe ratio per decision of 0.684, the ideal rule 0.682, always holding the asset 0.270.

One decision every 10 steps (right panel). The first split is on x in 86 datasets, never on a decoy; 14 trees keep no split. The median threshold is -0.46 ; half of them lie between -0.85 and -0.19 , and 90% between -1.53 and $+0.01$. Growth made 1.6 splits per tree and pruning left 1.04: 15 trees keep a second split on x (a finer position sizing), and 3 keep a second-level split on a pure-noise decoy. On test data: tree 0.724, ideal rule 0.742, rule at d 0.749, always the asset 0.628; the tree is 0.018 (standard error 0.004) below the ideal rule and 0.096 (0.005) above always holding the asset.

Why the spread is wide, and why it costs little. Only 48 of the 332 decisions used to grow the tree have x between -0.9 and 0 , and there the expected return is small (0 to 5%) against a noise of 13% per decision. The data cannot say where exactly inside this zone the threshold should be. For the same reason, the exact place matters little for performance.

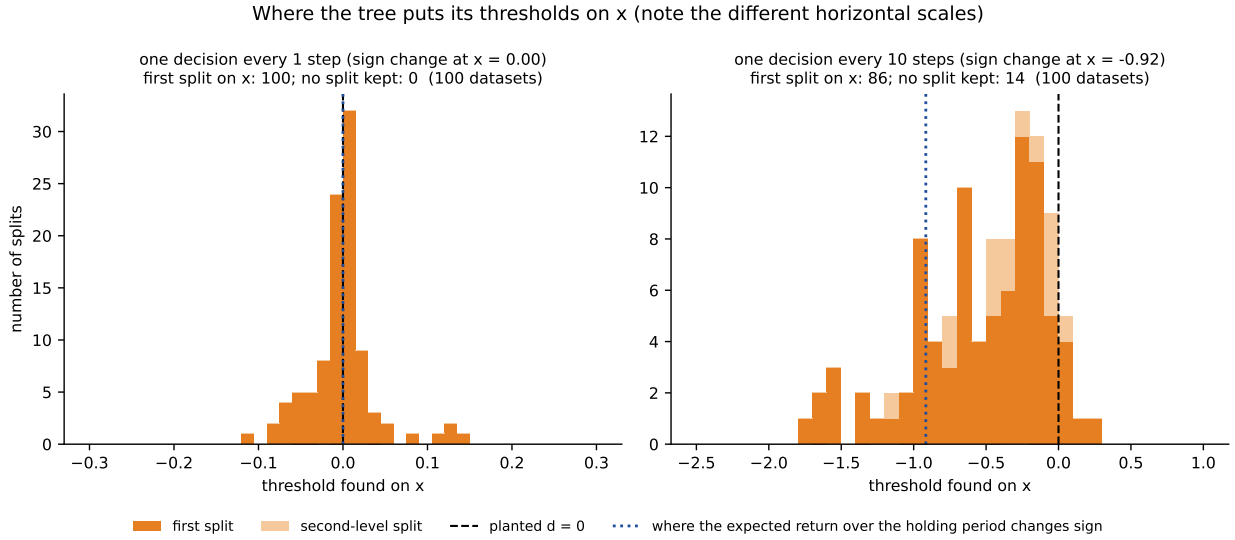


Figure 4: All thresholds on x kept in the final trees, over 100 independent datasets per panel. *Left:* one decision per step; the right target is d . *Right:* one decision every 10 steps; the right target is the zone between the sign change (dotted) and about -0.3 .

4.3 Effect of the speed of the feature and of the volatility of the price

We vary the reversion speed k of the feature and the volatility of the price, all else as in the base case, with 20 datasets per cell (Figures 5 and 6).

- **The true feature is found** (left map) in 95 to 100% of the datasets when a regime lasts at least about one holding period ($k \leq 0.1$) and the volatility is up to 20% per step, and still in 60 to 85% at 30% per step, where the drift is a tenth of the volatility. With $k = 0.2$ (a regime lasts half a holding period) it is found in 75 to 90% up to a volatility of 10%. With $k = 0.5$ (a regime lasts 2 steps) in 20 to 45%.
- **When x is not found, the usual outcome is no split at all** (middle map), not a wrong feature: over the 600 trees, a pure-noise decoy is the first split in 9 (1.5%) and the moving average in 30 (5%), almost all at volatilities of 20 to 30%. At $k = 0.5$ and low volatility 70 to 80% of the trees keep no split, and this is the right answer: the regime at the decision says almost nothing about the next 10 steps, and the ideal rule gains only 0.01% per decision over always holding the asset.
- **Share of the possible gain that is captured on test data** (right map): 96 to 100% for $k \leq 0.1$ at a volatility up to 3.2%, about 80% at 10%, and 12 to 40% at 20 to 30%.
- **The speed moves the target** (Figure 6). The faster the feature, the further below d the expected 10-step return changes sign: -0.04 , -0.05 , -0.16 , -0.37 , -0.92 and -3.6 for the six speeds. The tree's thresholds follow, staying between that point and d : medians -0.02 , $+0.02$, -0.04 , -0.19 , -0.47 and -1.52 .
- **The volatility changes the nature of the decision.** With λ fixed, a more volatile asset is no longer held fully: the ideal position becomes a gradual function of x over its whole range. A tree with two to four leaves replaces this slope by a few steps, so it captures a smaller share, and its best single threshold moves up towards the middle of the distribution of x (medians $+0.16$ to $+0.37$ for the slow features at volatilities of 10 to 30%). This is also why x is found slightly *more* often at 10% than at 1% when $k = 0.2$: the position depends on x over a wider range.

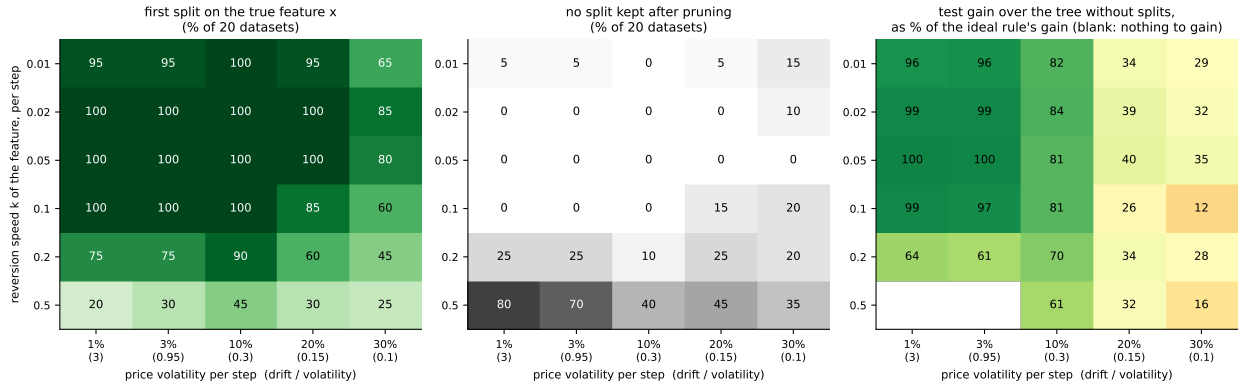


Figure 5: Twenty datasets per cell. Rows: reversion speed k of the feature (top: slow, a regime lasts many decisions; bottom: fast, the regime at the decision says little about the next 10 steps). Columns: volatility of the price per step, with the ratio drift/volatility in brackets.

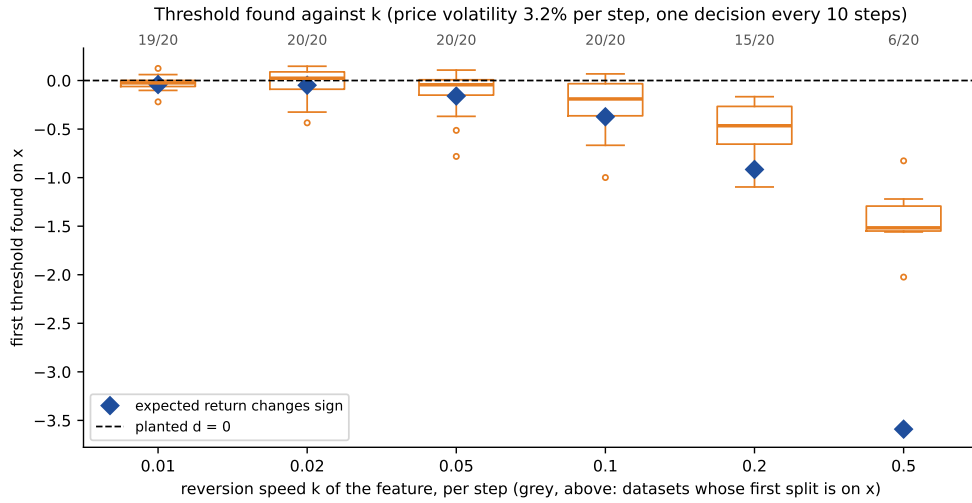


Figure 6: First threshold found on x , for each speed k , at the base volatility. Diamonds: where the true expected 10-step return changes sign.

5 What this test says, and what it does not

What it says.

1. **The mechanism works.** When the right answer is d , the tree recovers it to within a few hundredths of a standard deviation of x , with a single split, and ignores the three decoys.
2. **The tree learns where the decision changes, not where the regime changes.** With positions held for 10 steps it puts its threshold below d , where holding the asset starts to pay over the holding period, and it is blind to the jump of the expected return at d , because both sides of d lead to the same portfolio. To read the tree correctly: a threshold means “from here on, hold the asset”, not “here the drift changes”. This is the SPO principle of the articles, seen on a case where the two differ.
3. **Decoys are not a problem at this signal strength**, and pruning removes about a third of the splits that growth made.
4. **Two limits.** A feature much faster than the holding period leaves nothing to find, and the tree then rightly keeps no split. A high volatility with a fixed λ turns the problem into position sizing, which a small tree only approximates by steps.

What it does not say.

- The planted drift is enormous by market standards: 3% per step for a volatility of 3.2%, a ratio close to 1. The experiments of the main report used a ratio near 0.07 (0.2% a day for 3% a day), and there the splits were not reliably better than no split. In this note the ratio reaches 0.1 only through extreme volatilities. The natural next step is the same figures with small drifts, ratios of 0.02 to 0.1 at a volatility of 3%, which is what connects this test to real markets.
- There is one asset and cash, one informative feature, and regimes that depend on a single threshold.

6 How to reproduce

- `python recovery_test.py`: one dataset, printed in detail (5 seconds).
- `python recovery_figures.py run`: all experiments, 800 fits in parallel (about 14 minutes on this machine).
- `python recovery_figures.py plot`: the figures, written to `report/figures/`.
- The rows are checked by five unit tests (`tests/test_recovery.py`): alignment of features and returns, the covariance, and no look-ahead.